DSA01 – ASSIGNMENT 2 (CO2)
C++ Functions and Object-Oriented Programming

Name: A Shiva Shankar Reddy

Register Number:192425174

Department / Year: AIML/3rd year

Faculty Name: Dr Arun Kumar

GitHub Repository: https://github.com/shivareddy1012/DSA0102-192425174-C.Plus.Plus/tree/main/Assignment%202%20

Submission Date: 31 October 2026

Q1 – Call by Value, by Reference and by Pointer

Question and requirements

Problem context: Hostel Fee Concession Updater. Write the concession operation using value, reference and pointer parameters. Predict stored fees before running and verify. Explain parameter storage, why caller changes differ, when const reference is useful, and decide whether to modify the fee in place or return a new fee.

Also predict the exact output of the assignment's supplied change/bigger/show program before execution; complete TODO 1 in applyConcession; discuss the return-by-value alternative and time/space complexity.

Aim

To compare how value, reference and pointer parameters affect the caller's hostel fee.

Algorithm

1. Initialize three fees to 45000.
2. Apply a 10% concession using each parameter-passing method.
3. Print each stored fee.
4. Compare the results.

Program: Q1_concession.cpp

```
#include <iostream>
using namespace std;

void byValue(long long fee, int percent) {
    fee -= fee * percent / 100;
}
void byReference(long long& fee, int percent) {
    fee -= fee * percent / 100;
}
void byPointer(long long* fee, int percent) {
    *fee -= *fee * percent / 100;
}
int main() {
    long long fee1 = 45000, fee2 = 45000, fee3 = 45000;
    byValue(fee1, 10);
    cout << "After call by value: " << fee1 << '\\n';
    byReference(fee2, 10);
    cout << "After call by reference: " << fee2 << '\\n';
    byPointer(&fee3, 10);
    cout << "After call by pointer: " << fee3 << '\\n';
    return 0;
}
```

Expected output

After call by value: 45000\nAfter call by reference: 40500\nAfter call by pointer: 40500

Completed TODO program

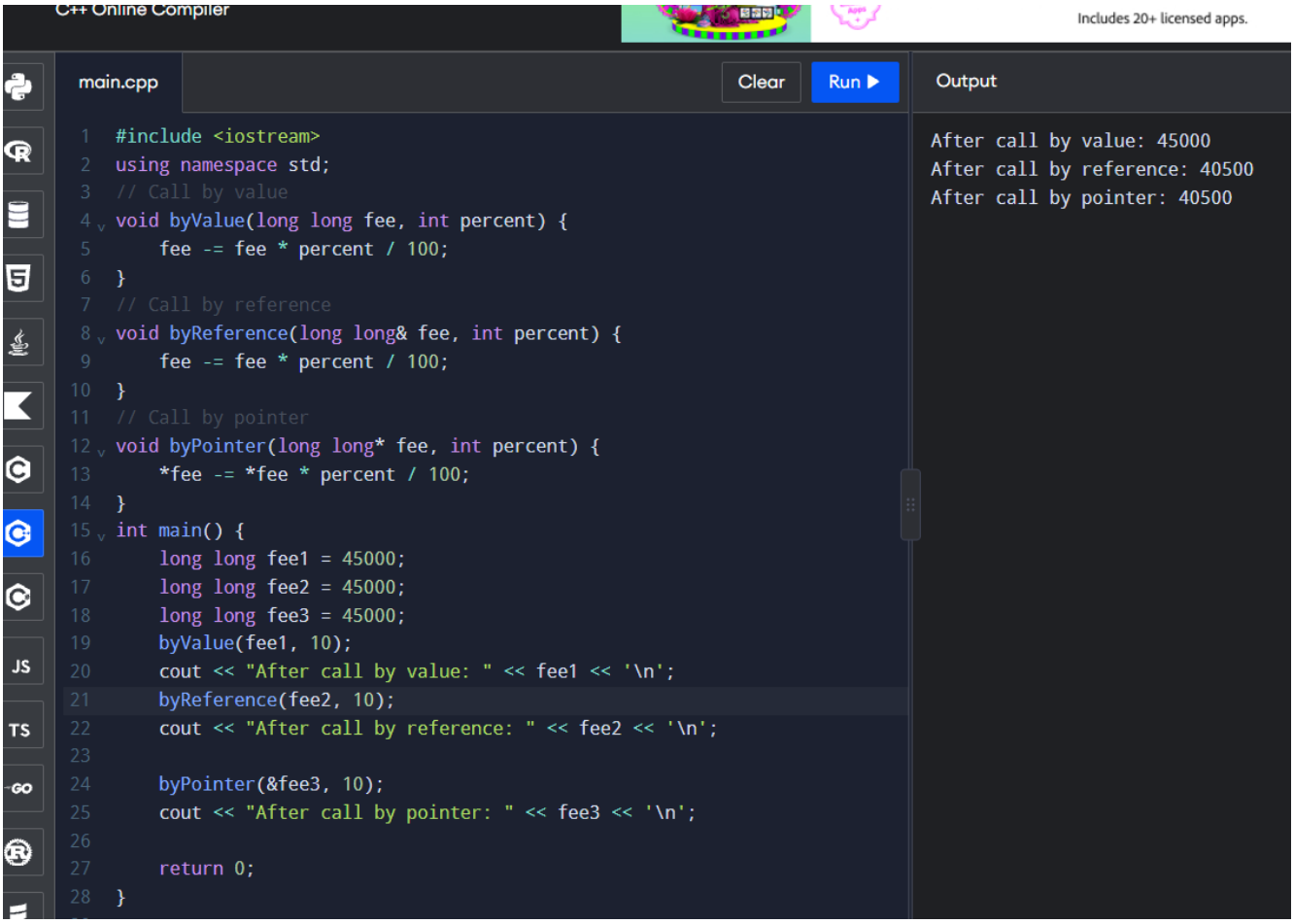
```
#include <iostream>
using namespace std;
void applyByValue(long long fee, int percent) {
   fee -= fee * percent / 100;
}
void applyConcession(long long& fee, int percent) {
   fee -= fee * percent / 100; // TODO completed
}
int main() {
   long long hostelFee = 45000;
   applyByValue(hostelFee, 10);
   cout << "after call by value   : " << hostelFee << '\\n';
   applyConcession(hostelFee, 10);
   cout << "after call by reference: " << hostelFee << '\\n';
}
```

Expected:\nafter call by value   : 45000\nafter call by reference: 40500

Supplied prediction program

```
#include <iostream>
void change(int a, int& b, int* c) {
   a += 10; b += 10; *c += 10;
}
int& bigger(int& p, int& q) { return p > q ? p : q; }
void show(int x, int y = 5, int z = 10) {
   std::cout << x + y + z << ' ';
}
int main() {
   int a = 1, b = 2, c = 3;
   change(a, b, &c);
   std::cout << a << ' ' << b << ' ' << c << '\\n';
   bigger(a, c) = 100;
   std::cout << a << ' ' << c << '\\n';
   show(1); show(1, 2); show(1, 2, 3);
   std::cout << '\\n';
}
```

output:



Inquiry / design answers

- Value: parameter is a separate copy; reference: alias to original; pointer: address is copied and dereferenced to modify original.
- Reference and pointer can change caller data; value changes only the local copy.
- Use const T& for read-only access to potentially large objects without copying.
- For a simple fee calculation, returning the new fee is often easier to test and avoids hidden mutation. In-place reference is reasonable when mutation is explicitly desired.

Alternative and complexity

```
long long applyConcession(long long fee, int percent) {
    return fee - fee * percent / 100;
}
// usage: fee = applyConcession(fee, 10);
```

Time: O(1) for all scalar implementations. Passing long long by value copies one scalar; a reference avoids copying the referred object; a pointer copies an address. A large object's by-value copy can cost proportional to its contents; const reference avoids that copy. Exact implementation costs depend on type/compiler.

Q2 – Return by Reference and Its Dangers

Question and requirements

Implement a function returning a reference to the highest mark in an array, allowing the caller to modify it. Also demonstrate why returning a reference to a local variable is unsafe. Explain reference vs const reference vs copy; vector reallocation; index/pointer alternatives; O(n) search time and O(1) auxiliary space.

Aim and algorithm

Find the highest mark's element and return it by reference; separately show the unsafe pattern only as commented explanation, never call it.

5. Start with a marks array.
6. Find the index of the maximum element.
7. Return the array element by reference.
8. Update that element through the returned reference and print the array.

Program: Q2_marks.cpp

```
#include <iostream>
using namespace std;

int& highestMark(int marks[], int n) {
    int best = 0;
    for (int i = 1; i < n; ++i)
        if (marks[i] > marks[best]) best = i;
    return marks[best];
}

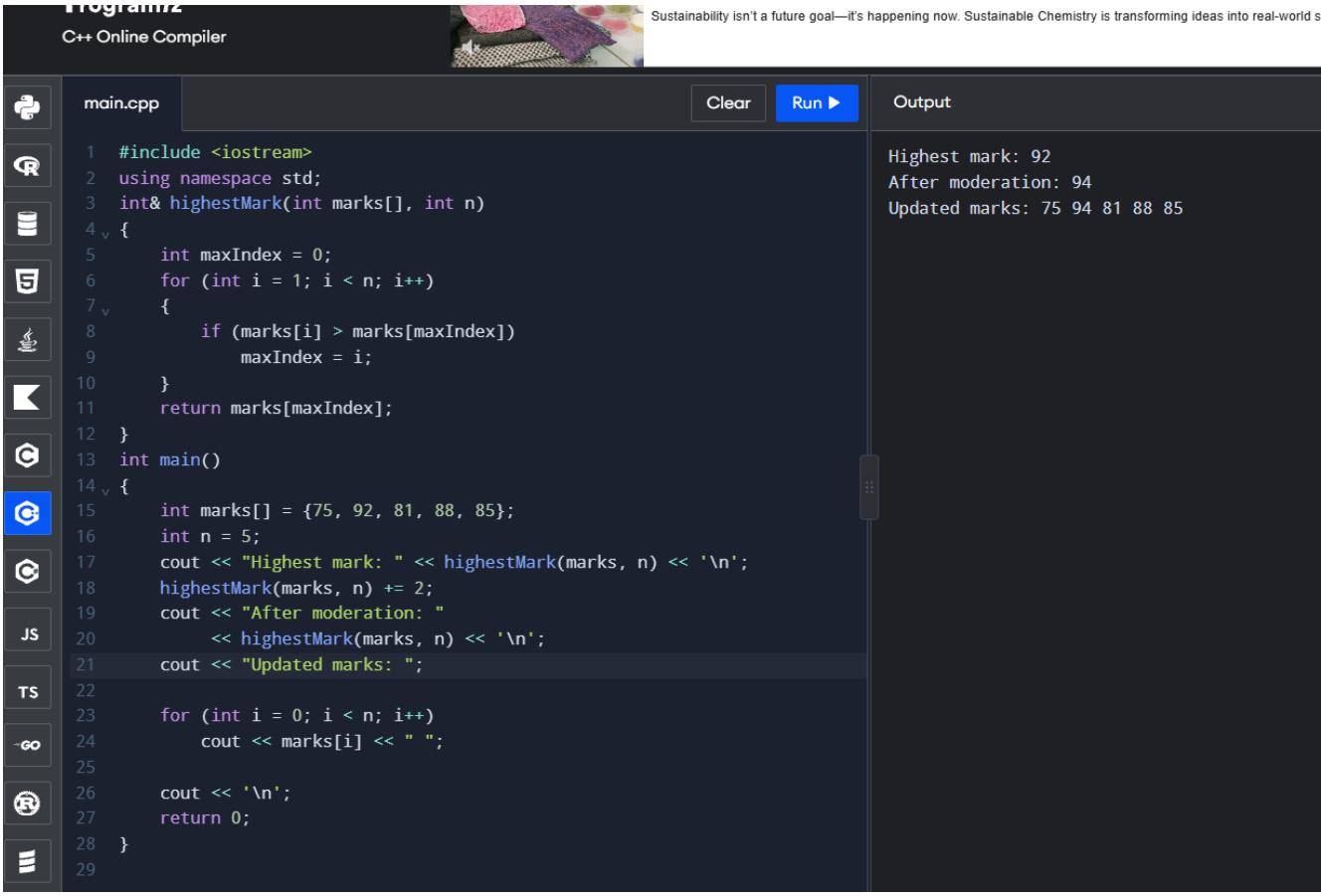
// UNSAFE EXAMPLE — do not call:
// int& badReference() {
//     int x = 10;
//     return x; // dangling reference; x dies at function exit
// }

int main() {
    int marks[] = {76, 91, 84, 88};
    highestMark(marks, 4) += 2;
    for (int x : marks) cout << x << ' ';
    cout << '\\n';
    return 0;
}
```

Expected output

```
76 93 84 88
```

Output:



Explanation / alternative

- Returning a reference to an array element permits assignment through the call, because the element continues to exist while the array exists.
- Returning a reference to a local variable leaves a dangling reference after the function returns; using it is undefined behavior. A compiler may warn.
- int& permits modification; const int& prevents modification through that reference; int returns a copy independent of the array.
- A vector can reallocate when it grows, invalidating references/pointers/iterators to its elements. Obtain a fresh reference after growth.
- Index alternative: return best index and use marks[index]. Pointer alternative: return &marks[best] while storage remains valid.

Complexity: finding maximum is O(n) time and O(1) auxiliary space for reference, index, or pointer result. Index can be safer to store across vector growth if reacquired after growth; a stored index remains meaningful if element ordering isn't changed.

Evidence to add

- Program output screenshot
- Compiler warning evidence for unsafe local-reference example (compile separately if demonstrating)

Q3 – Default Arguments and Function Overloading

Question and requirements

Explain ambiguity among overlapping charge(double), charge(double,bool), and charge(double,bool=false,bool=false) overloads. Design unambiguous functions supporting grams as int, kilograms as double, and Parcel objects. Predict six calls including charge(2) and charge(2.0f). Explain overload selection and right-to-left defaults; compare a Parcel structure/template alternative.

Design note

Use distinct function names for unit variants to eliminate ambiguity and clearly identify units. A separate Parcel overload accepts a Parcel.

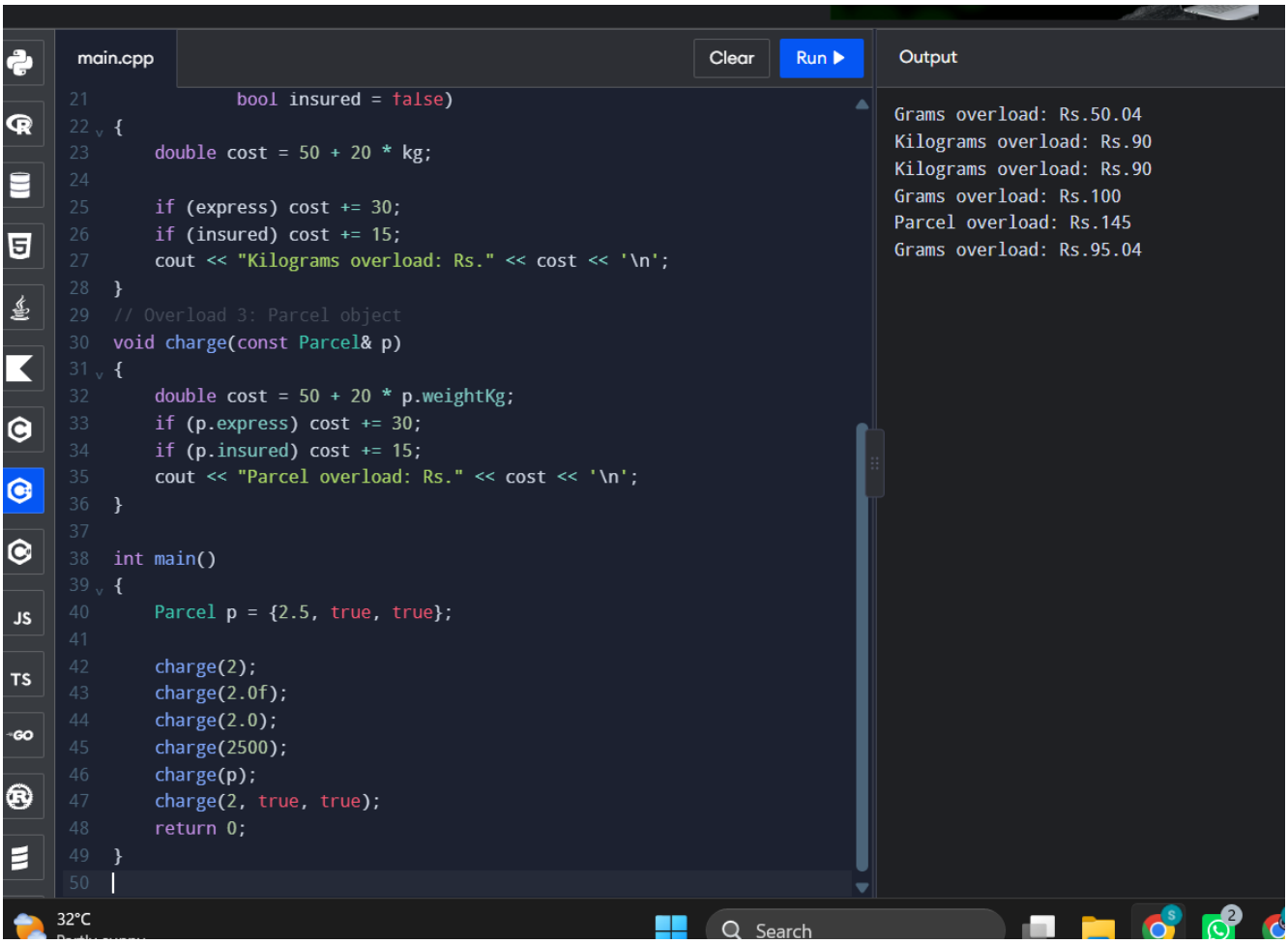
Program: Q3_courier.cpp

```
#include <iostream>
using namespace std;

struct Parcel {
   double kg;
   bool express;
   bool insured;
};
double chargeKg(double kg, bool express = false, bool insured = false) {
   double cost = 50.0 + kg * 20.0;
   if (express) cost += 30.0;
   if (insured) cost += 15.0;
   return cost;
}
double chargeGrams(int grams, bool express = false, bool insured = false) {
   return chargeKg(grams / 1000.0, express, insured);
}
double chargeParcel(const Parcel& p) {
   return chargeKg(p.kg, p.express, p.insured);
}
int main() {
   Parcel p{2.0, true, false};
   cout << chargeKg(2) << '\\n';          // kg function
   cout << chargeKg(2.0f) << '\\n';       // kg function
   cout << chargeGrams(500) << '\\n';     // grams function
   cout << chargeKg(1.5, true) << '\\n';
   cout << chargeKg(1.5, false, true) << '\\n';
   cout << chargeParcel(p) << '\\n';
}
```

The original overload set is ambiguous because one-argument charge(2) can match several overloads when defaults/overloads overlap. Avoid overlapping signatures or use distinct names.

Output:



Six-call selection

- chargeKg(2): chargeKg(double,bool,bool), with both defaults.
- chargeKg(2.0f): chargeKg(double,bool,bool), float converts to double.
- chargeGrams(500): grams overload, because function name identifies units.
- chargeKg(1.5,true): kg overload, insured defaults false.
- chargeKg(1.5,false,true): kg overload with all arguments.
- chargeParcel(p): Parcel overload.

Default arguments must be supplied from the right so trailing omitted parameters have values. Overload resolution favors exact match, then promotion, then conversion, but defaults do not themselves create a distinct overload.

Alternative and complexity

A single Parcel structure with optional fields makes future options easier to add but requires constructing/configuring the object. A template may help generic types but requires careful constraints. Overload resolution occurs at compile time with no runtime dispatch cost. Charge calculation is O(1). Passing Parcel by const reference avoids copying.

Evidence to add

- Your six-call prediction before execution
- Observed output screenshot
- Explanation of ambiguity and selection

Q4 – Specifying a Class with an Array Inside It

Question and requirements

Specify SavingsAccount with private account number, balance, array of the latest 10 transactions and count; public deposit(), withdraw(), miniStatement(). Define some methods inside and some outside using scope resolution. Explain private array, shifting versus circular index for the 11th transaction, scalability to 50 records, and compare fixed array/deque/circular buffer.

Program: Q4_account.cpp

```
#include <iostream>
#include <string>
using namespace std;

class SavingsAccount {
   string accountNo;
   double balance;
   string transactions[10];
   int count = 0, next = 0;
public:
   SavingsAccount(string no, double opening) : accountNo(no), balance(opening) {}
   void addTransaction(const string& text) {
      transactions[next] = text;
      next = (next + 1) % 10;
      if (count < 10) ++count;
   }
   void deposit(double amount) {
      if (amount <= 0) return;
      balance += amount;
      addTransaction("Deposit: +" + to_string(amount));
   }
   void withdraw(double amount);
   void miniStatement() const;
};

void SavingsAccount::withdraw(double amount) {
   if (amount <= 0 || amount > balance) return;
   balance -= amount;
   addTransaction("Withdraw: -" + to_string(amount));
```

```
}
void SavingsAccount::miniStatement() const {
   cout << "Account: " << accountNo << "\\nBalance: " << balance << '\\n';
   cout << "Recent transactions (oldest to newest):\\n";
   int start = (count == 10) ? next : 0;
   for (int i = 0; i < count; ++i)
      cout << transactions[(start + i) % 10] << '\\n';
}
int main() {
   SavingsAccount a("SA101", 1000);
   a.deposit(500); a.withdraw(200);
   a.deposit(100); a.withdraw(50);
   a.miniStatement();
}
```

Output note

Output includes the account number, balance and four transaction records. Exact decimal formatting from to_string typically displays six fractional digits. Capture your compiled output as evidence.

Design explanation

- Private data protects the account's invariants from invalid external edits.
- Shifting a full array on each new entry costs O(k), where k is the retained transaction count; circular indexing inserts in O(1).
- Circular array storage uses O(k) fixed memory and can retain the latest 10 or 50 by changing capacity and indexing logic.
- std::deque can push_back and pop_front efficiently (typically O(1)); it uses O(k) storage plus container overhead.

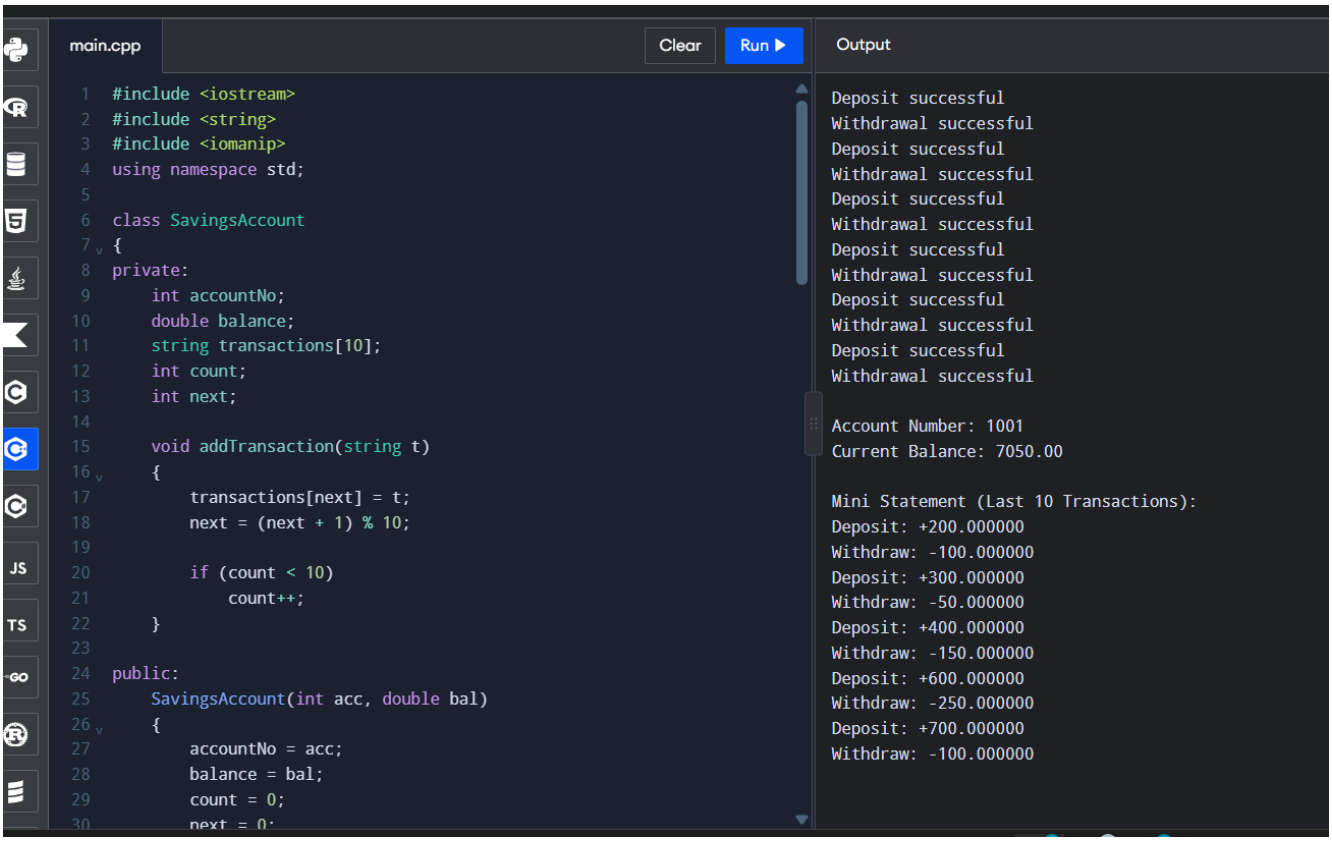
Complexity

Deposit/withdraw update and circular insertion: O(1). Mini-statement: O(k) to print the stored entries. Storage: O(k).

Evidence to add

- Output screenshot
- Test the 11th transaction and show that only the newest 10 remain
- Discuss how design changes for 50 records

Output for 4:



Q5 – Static Members and Arrays of Objects

Question and requirements

Design BusTicket using static shared state and a fixed capacity. Assign next seat while capacity remains; excess tickets wait. Explain static member definition, static function limitations, array of objects, and count behavior. Complete TODOs and predict the Token output.

Program: Q5_busticket.cpp

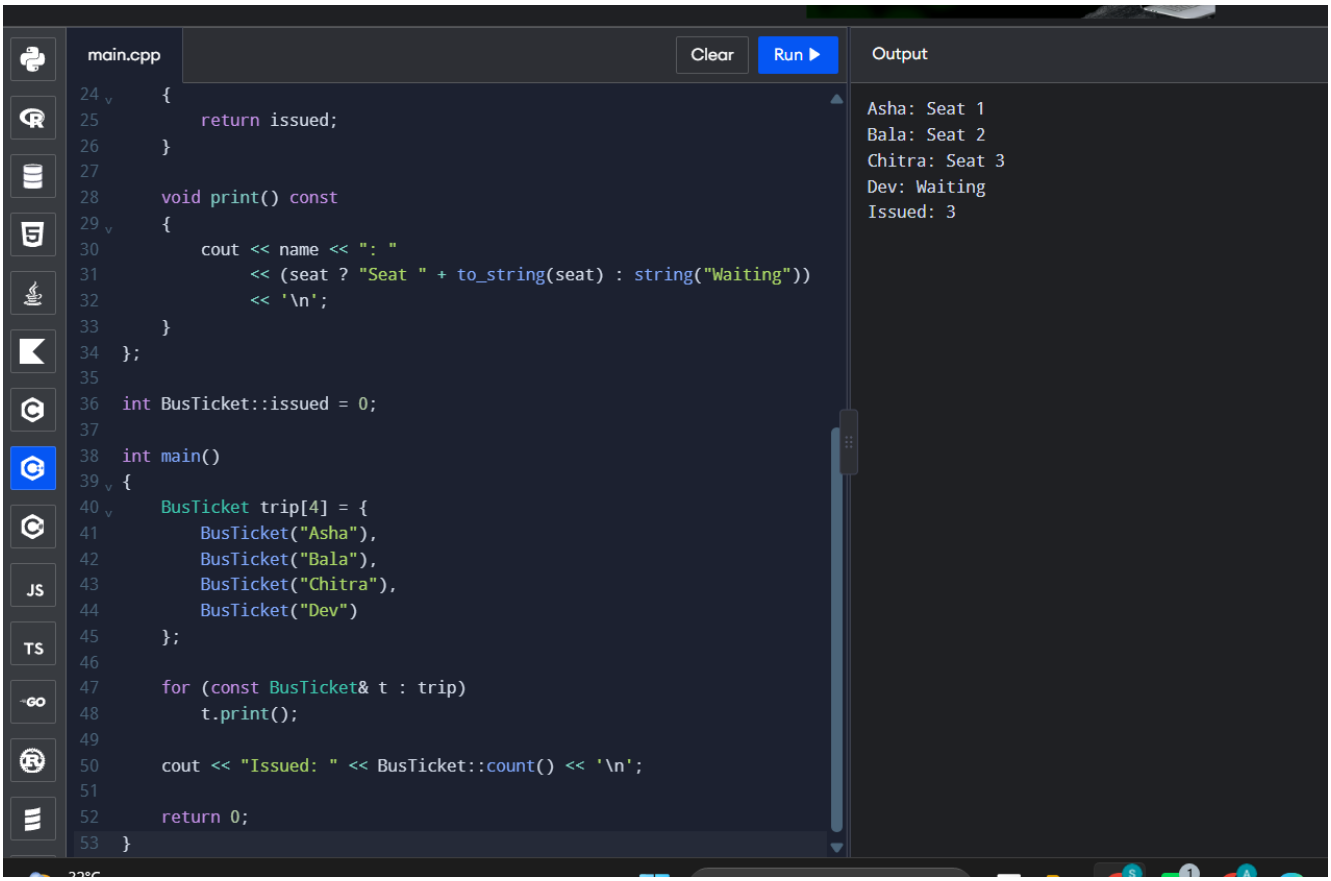
```
#include <iostream>
#include <string>
using namespace std;
class BusTicket {
   static int issued;
   static const int CAPACITY = 3;
   int seat;
   string name;
public:
   explicit BusTicket(const string& n) : seat(0), name(n) {
      if (issued < CAPACITY) seat = ++issued;
   }
   static int count() { return issued; }
   void print() const {
```

```
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat) : string("Waiting"))
             << '\\n';
    }
};
int BusTicket::issued = 0;
int main() {
    BusTicket trip[4] = {BusTicket("Asha"), BusTicket("Bala"),
                         BusTicket("Chitra"), BusTicket("Dev")};
    for (const BusTicket& t : trip) t.print();
    cout << "Issued: " << BusTicket::count() << '\\n';
}
```

Expected output

Asha: Seat 1\nBala: Seat 2\nChitra: Seat 3\nDev: Waiting\nIssued: 3

Output:



Supplied Token prediction

```
#include <iostream>
class Token {
    static int next;
    int id;
public:
```

```
    Token() { id = ++next; }
    int get() const { return id; }
    static int issued() { return next - 100; }
};
int Token::next = 100;
int main() {
    Token counter[3];
    Token extra;
    std::cout << counter[0].get() << ' ' << counter[2].get()
              << ' ' << extra.get() << ' ' << Token::issued() << '\\n';
}
```

Predicted output: 101 103 104 4

Explanation: static next starts at 100. Three array objects and one extra object are constructed, incrementing it four times. Static count is shared among all objects. A static member function has no this pointer and cannot directly access an object's non-static fields.

If tickets are created inside a function called twice, static issued persists between calls; once capacity is reached, later tickets wait. This current design models one bus only. For two buses, use a TicketOffice/Bus-specific counter or separate static state per bus identifier.

Alternative and complexity

A global counter is simple but weakly encapsulated. A TicketOffice object can hold capacity and count, enabling multiple buses and explicit ownership. Issuance is O(1), with O(1) shared counter storage per office/bus.

Evidence to add

- Both programs' output screenshots
- Test array larger than capacity
- Test creation across repeated function calls

Q6 – Friend Functions and Inline Functions

Question and requirements

Design comparison for a Time class with private hour/minute. Use friend functions to calculate minutes between times and determine earlier time. Explain friend vs member, inline meaning, why private data should stay private. Test around noon and equal times. Measure 10,000,000 calls with/without inline using -O0 and -O2; compare alternatives.

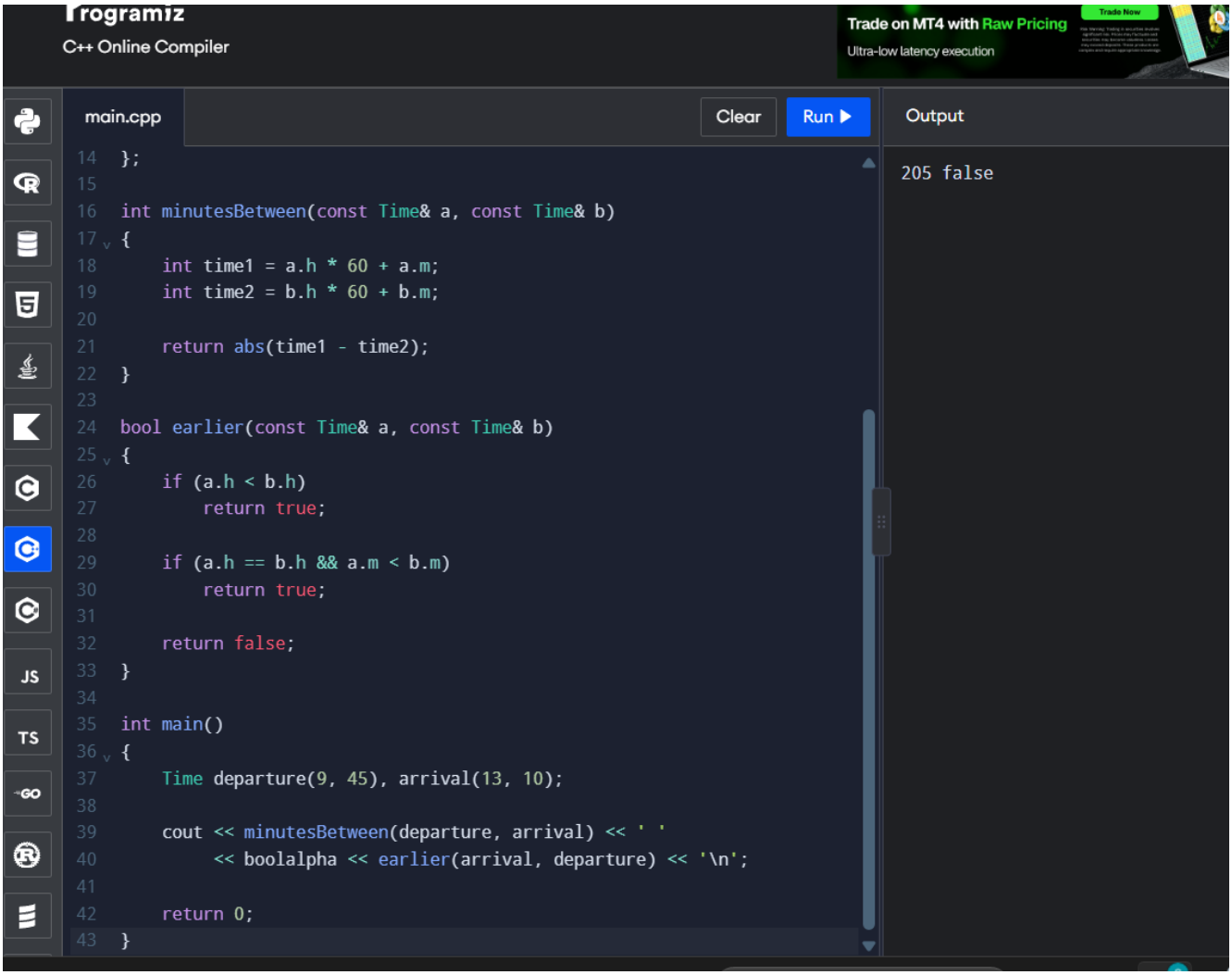
Program: Q6_time.cpp

```
#include <iostream>
#include <cstdlib>
using namespace std;
class Time {
   int h, m;
public:
   Time(int hh, int mm) : h(hh), m(mm) {}
   friend int minutesBetween(const Time& a, const Time& b);
   friend bool earlier(const Time& a, const Time& b);
};
inline int minutesBetween(const Time& a, const Time& b) {
   int x = a.h * 60 + a.m, y = b.h * 60 + b.m;
   return abs(x - y);
}
inline bool earlier(const Time& a, const Time& b) {
   return a.h * 60 + a.m < b.h * 60 + b.m;
}
int main() {
   Time departure(9, 45), arrival(13, 10);
   cout << minutesBetween(departure, arrival) << ' '
        << boolalpha << earlier(arrival, departure) << '\\n';
   Time noonBefore(11, 59), noonAfter(12, 1);
   cout << minutesBetween(noonBefore, noonAfter) << ' '
        << earlier(noonBefore, noonAfter) << '\\n';
   cout << minutesBetween(noonBefore, noonBefore) << ' '
        << earlier(noonBefore, noonBefore) << '\\n';
}
```

Expected output

```
205 false\n2 true\n0 false
```

Output:



Explanation

- Friend functions are non-members but are permitted to access private fields; call them as minutesBetween(a,b). A member function is called through an object and has this.
- Inline is a request/permission for compiler substitution/linkage behavior, not a guarantee that machine code will be inlined. Optimization choices are compiler-dependent.
- Public fields expose representation and permit invalid values; keep fields private and enforce valid hour/minute inputs if expanding this design.
- Member functions or public getters can implement the same O(1) logic. Getters improve encapsulation but add interface code; compiler optimization may remove overhead.

Timing investigation

Use a loop with 10,000,000 calls and a checksum so the compiler cannot remove the work. Build variants with and without inline, at -O0 and -O2. Record measurements from your own computer; do not invent timings.

g++ -std=c++17 -O0 Q6_time.cpp -o time_O0\ng++ -std=c++17 -O2 Q6_time.cpp -o time_O2

Timing table to fill:

Variant | Optimization | Measured duration\nWithout inline | -O0 | _____\nWith inline | -O0 | _____\nWithout inline | -O2 | _____\nWith inline | -O2 | _____

Time per comparison is O(1). The test's loop is O(n) for n calls. Report your actual timing and machine/compiler details.

Evidence to add

- Correctness test screenshots
- Compiler/version
- Four measured timings and exact commands
- Evidence-based choice of friend/member/getter

Q7 – Diagnose, Redesign and Validate

Selected Redesign: Q4 – Savings Account Transaction History

A. Diagnose the Limitation

The original SavingsAccou1nt program stores the last 10 transactions in a fixed array. When the array becomes full, older transactions must be removed to store a new transaction.

If the program uses the shifting method, every new transaction requires shifting the remaining transactions one position to the left before inserting the latest transaction.

For a history capacity of k, inserting a transaction when the array is full requires O(k) time due to shifting. This can become inefficient when the required transaction history increases from 10 to 50, 500 or more transactions.

Therefore, the main limitation is the repeated shifting of transaction records.

B. Define the Design Objective (2 Marks)

The objective is to redesign the transaction history using a circular buffer.

The measurable objectives are:

1. Reduce the insertion time from O(k) in the shifting design to O(1) in the circular-buffer design.

2. Retain only the latest k transactions.

3. Display all retained transactions in chronological order.

4. Ensure both designs produce the same transaction history for identical inputs.

C. Design the Revised Solution (4 Marks)

Original Design – Shifting Array

When the array is full:

1. Shift transactions from index 1 to index k-1 one position to the left.
2. Insert the new transaction at index k-1.
3. Discard the oldest transaction.

Redesigned Solution – Circular Buffer

The circular buffer uses a fixed-size array, a count variable and a next index.

When a transaction arrives:

1. Store the new transaction at buffer[next].
2. Update next = (next + 1) % capacity.
3. Increase count until it reaches the capacity.

When the buffer is full, the next insertion automatically overwrites the oldest transaction.

Class-level design:

SavingsAccount

- accountNo
- balance
- transactions[k]

- count

- next

+ deposit()

+ withdraw()

+ addTransaction()

+ miniStatement()

Circular Buffer Pseudocode

```
insert(item):
    buffer[next] = item
    next = (next + 1) % capacity
    count = min(count + 1, capacity)
miniStatement():
    start = (next - count + capacity) % capacity
    for i = 0 to count - 1:
        print buffer[(start + i) % capacity]
```

Memory and Trade-offs

- Both designs require O(k) space to store k transactions.
- The shifting design performs O(k) work for each insertion when full.
- The circular buffer performs O(1) insertion work.
- The circular buffer requires extra index management and careful chronological printing.
- If the bank increases the capacity, the circular-buffer design needs only a capacity change, provided the buffer storage is allocated accordingly.

D. Predict Before Implementing (1 Mark)

I predict that the circular-buffer design will perform better than the shifting-array design when the transaction history is full.

The shifting design requires O(k) operations per insertion, whereas the circular-buffer design requires O(1) operations per insertion. Both designs should retain the same latest k transactions and display them in chronological order.

E. Implement and Validate the Redesign (2 Marks)

Two implementations are used for comparison:

- Original: Fixed array with shifting.
- Redesigned: Fixed circular array.

Both implementations will receive the same transaction sequence.

Testing procedure:

1. Test capacities of 10, 50, 500 and 5000 transactions.
2. Insert 200,000 transactions into each implementation.

3. Measure the insertion time of both programs using the same compiler and environment.
4. Compare the retained transactions.
5. Verify that both implementations retain the same latest k transactions in chronological order.

Validation criteria:

- Both implementations must retain the latest k transactions.
- Both must display transactions in the same chronological order.
- The insertion time should be measured for both designs.
- The actual results should be recorded after execution.

Comparison Table

Capacity (k)	Shifting Time	Circular Buffer Time
10	Record after execution	Record after execution
50	Record after execution	Record after execution
500	Record after execution	Record after execution
5000	Record after execution	Record after execution

The measured execution times and correctness results will be filled in after running the benchmark program.

F. Defend the Final Design (2 Marks)

The circular-buffer design is preferred because it avoids repeatedly shifting transaction records. It provides constant-time insertion and is more suitable when the transaction history capacity increases.

Both designs require O(k) memory, but the circular buffer reduces the work required for inserting a transaction when the history is full.

The circular buffer introduces slightly more index-management complexity. However, its performance advantage makes it a better choice for maintaining transaction history.

The original shifting-array design may still be suitable for very small histories where simplicity is more important than insertion performance.

Conclusion: The circular-buffer redesign is expected to improve insertion efficiency while maintaining the same transaction history. The final performance improvement will be confirmed using the measured benchmark results.

Implementation code for Q7 validation

```
#include <iostream>

#include <vector>

#include <string>

#include <chrono>

using namespace std;

using namespace chrono;

class ShiftingBuffer

{

   vector<string> data;

   int capacity, count;

public:

   ShiftingBuffer(int k)

      : data(k), capacity(k), count(0) {}


   void insert(const string& item)

   {
```

```
        if (count < capacity)
        {
            data[count++] = item;
        }
        else
        {
            for (int i = 0; i < capacity - 1; i++)
                data[i] = data[i + 1];
            data[capacity - 1] = item;
        }
    }
    vector<string> history() const
    {
        return vector<string>(
            data.begin(), data.begin() + count);
    }
};
class CircularBuffer
{
    vector<string> data;
    int capacity, count, next;
public:
    CircularBuffer(int k)
        : data(k), capacity(k), count(0), next(0) {}

    void insert(const string& item)
    {
        data[next] = item;
        next = (next + 1) % capacity;
```

```
        if (count < capacity)
            count++;
    }
    vector<string> history() const
    {
        vector<string> result;
        int start = (next - count + capacity) % capacity;
        for (int i = 0; i < count; i++)
            result.push_back(data[(start + i) % capacity]);
        return result;
    }
};
int main()
{
    const int operations = 200000;
    int capacities[] = {10, 50, 500, 5000};

    for (int k : capacities)
    {
        ShiftingBuffer original(k);
        CircularBuffer redesigned(k);
        auto start1 = steady_clock::now();

        for (int i = 1; i <= operations; i++)
            original.insert("Transaction " + to_string(i));
        auto end1 = steady_clock::now();
        auto start2 = steady_clock::now();
        for (int i = 1; i <= operations; i++)
            redesigned.insert("Transaction " + to_string(i));
        auto end2 = steady_clock::now();
```

```
    auto t1 = duration_cast<microseconds>(
        end1 - start1).count();
    auto t2 = duration_cast<microseconds>(
        end2 - start2).count();
    cout << "\nCapacity: " << k << '\n';
    cout << "Shifting time: " << t1 << " microseconds\n";
    cout << "Circular time: " << t2 << " microseconds\n";
    if (original.history() == redesigned.history())
        cout << "Validation: PASSED\n";
    else
        cout << "Validation: FAILED\n";
  }
  return 0;
}
```

Output:

